Assignment 2

For this assignment, you may work with in groups of 2 or 3. This assignment should provide you with experience in the design and analysis of algorithms using basic data structures.

1. [12 marks] Write a program in C, which simulates the well-known **move-to-front strategy** which is useful for caching, data compression, and many other applications where items that have been recently accessed are more likely to be re-accessed. Your program should read in integers from a file and maintain the numbers in a list **with no duplicates**. When you read in a previously unseen integer, insert it at the front of the list. When you read in a duplicate integer (which has been previously inserted), delete it and reinsert it at the beginning of the list.

   Requirements:
   - Your list must be implemented using a doubly-linked list. You are given List.h to help you get started. Some functions have been implemented, and some you will need to code. Follow the instructions given in the comments.
   - The move-to-front strategy should be implemented in moveToFront.c and starter code has been provided. You will need to write code to read elements from the file and insert them into a list according to the move-to-front strategy described above. *Your algorithm must use the functions defined in List.h.*
   - Your code must be clear and readable (*i.e., properly indented and documented throughout where necessary*)

2. [2 marks] Run the program on different values of *n* by doing the following.

   a. Read in the first 50,000 entries only found in "test_dat.txt"
   b. Read in the first 100,000 entries only found in "test_dat.txt"
   c. Read in the first 150,000 entries only found in "test_dat.txt"
   d. Read in the first 200,000 entries only found in "test_dat.txt"

   Record the time the algorithm takes for each input size. For each *n*, you must run your program 5 times. Summarize your results by providing a table like that shown on the next page*:*

Input Size	Run #1	Run #2	Run #3	Run #4	Run #5	Average Execution Time (ms)
50000						
100000						
150000						
200000						

3. [2 marks] Given how you implemented the move-to-front strategy, determine the theoretical complexity of your algorithm with respect to the input size in both the best and worst-case scenarios. Express using Big-O notation *giving the tightest bound possible.* Make sure to explain your reasoning by including a discussion of the complexity of the functions in List.h that your algorithm uses.

4. [3 marks] Given your average execution times, determine the growth rate of time with respect to the input size. (*See #3 of Assignment #1.*) Discuss whether it is consistent with the theoretical growth rate that you determined in the previous question.

5. [1 mark] Discuss how the growth rate, as determined from execution times, would compare to that obtained if the input file contained no duplicates. *(You may use the attached* NoDuplicates.txt *file to test your hypothesis.)*

What to include in your submission:

- One .zip folder **per group**, containing only:
  - List.h and moveToFront.c
  - A PDF containing your answers to #2 - #5.

Please note:

- You are expected to discuss every aspect of the assignment as a group. Each group member must be prepared to demo/answer questions relating to any part of the assignment.
- The submission folder should be named Assignment2_<*last names of group members separated by '_'*>.zip. For example, Hoda Fahmy and Dipak Pudasaini's submission for the second assignment would be named *Assignment2_Fahmy_Pudasaini.zip*.
- Make sure each file clearly states the names of the students in your group. Source code must include a program header *(i.e., a block comment at the top with your names, date, description of program, etc.).*
- You will each be given the same mark.
- You are not tied to working with the same group as Assignment #1.